

RSA: 非对称加密算法的一种
密钥对 { 公钥: 加密密钥, 公开
私钥: 解密密钥, 保密 }

费马小定理: 两个大质数相乘很容易, 但对其
乘积 (即同构解) 很难

→ 乘积为公钥, 两个大质数组成私钥

生成密钥对:

① 选择两个大质数 p 和 q (p, q 互质)

② 计算 $N = pq$

对于现在的计算机, 密钥 n 的长度为
1024 比特大, 甚至不可能从公钥推
出私钥信息。

③ 欧拉函数 $\varphi = (p-1)(q-1)$

↓
不大于 n 且与 n 互质的数是 $(p-1)$

和 $(q-1)$ 的乘积即 $\varphi(n) = (p-1)(q-1)$

④ 选择一个整数 E 满足:

$$1 < E < \varphi$$

且 E 和 φ 互质

⑤ 计算 E 关于 φ 的模逆元 D

即 $E \times D = 1 \pmod{\varphi}$

2) 公钥 $PK = (E, N)$ $\rightarrow$ 公钥指数
私钥 $SK = (D, N)$ $\rightarrow$ 私钥指数

RSA 加密和解密

给定明文 M , 加密:

密文 C : $C = M^E \pmod{N}$

给定密文 C , 解密

明文 M : $M = C^D \pmod{N}$

数论知识延伸：模的逆元的优化算法

① 扩展欧几里得算法：

对 $ax + by = \gcd(a, b)$ 方程
($\gcd(a, b)$ 是 a 与 b 的最大公约数)

当模的逆元 D 时

$$Ex + y = \gcd(E, \varphi) = 1$$

即 x 为模的逆元 D

$$ED + \varphi y = 1$$

该方程有解组 (D, y)

② 费马小定理：如果 p 是质数， a 是任意一个不被 p 整除的整数，则 $a^{p-1} = 1 \pmod{p}$

$$\text{即 } E \cdot E^{p-2} = 1 \pmod{\varphi}$$

$$\text{则模的逆元 } D = E^{p-2} \pmod{\varphi}$$

CRT中国剩余定理 加速 RSA 算法

p, q, N 的含义所指

对 $\forall m_1, m_2, 0 \leq m_1 < p, 0 \leq m_2 < q$

有且仅有一个 $m, 0 \leq m < N$, 使得

$$m_1 = m \bmod p$$

$$m_2 = m \bmod q$$

则对于解密 RSA 的流程

$$M = C^D \bmod N, \text{ 可解为}$$

$$M_1 = C^D \bmod p$$

$$M_2 = C^D \bmod q$$

然后计算 M

实现了模 N 降为 p, q

再来降 D

$$\text{令 } D = K(p-1) + r$$

$$\text{So } C^D \bmod p = C^{K(p-1)+r} \bmod p$$

$$= C^r \cdot C^{K(p-1)} \bmod p$$

$$\therefore C^{(p-1)} \bmod p = 1$$

$$\therefore C^D \bmod p = C^r \bmod p$$

$$r = D \bmod (p-1)$$

$$\text{可得 } D_p = D \bmod (p-1)$$

$$D_q = D \bmod (q-1)$$

$$M_1 = C^{D_p} \bmod p$$

$$M_2 = C^{D_q} \bmod q$$

$$q \text{ 对 } p \text{ 的逆元 } q_{inv} = q^{-1} \bmod p$$

最后的明文为：

$$H = q_{inv} (M_1 - M_2) \bmod p$$

$$M = M_2 + Hq \bmod p \times q$$

大数的幂运算

同余定理: $(a \times b) \% C = ((a \% C) \times (b \% C)) \% C$

$$\text{则 又 对 } a^b \% C = (a \% C)^b \% C$$

快速幂算法: 将十进制数 b 转换为二进制

$$b_{(10)} = a_{n-1} a_{n-2} \dots a_1 a_0_{(2)}$$

$$\text{则 } b = a_{n-1} \cdot 2^{n-1} + a_{n-2} \cdot 2^{n-2} + \dots + a_1 \cdot 2 + a_0$$

$$\begin{aligned} a^b &= a^{a_{n-1} \cdot 2^{n-1}} + a^{a_{n-2} \cdot 2^{n-2}} + \dots + a^{a_1 \cdot 2} + a^{a_0} \\ &= a^{a_{n-1} \cdot 2^{n-1}} \cdot a^{a_{n-2} \cdot 2^{n-2}} \cdot \dots \cdot a^{a_1 \cdot 2} + a^{a_0} \end{aligned}$$

$$\text{则 } a^b \bmod C = (a^{a_{n-1} \cdot 2^{n-1}} \bmod C) \cdot \dots$$

又 因 $a_i \in \{0, 1\}$, 设 $a_{k_i} = 1$

$$\begin{aligned} \text{则 } a^b \bmod C &= (a^{2^{k_n}} \bmod C) \cdot (a^{2^{k_{n-1}}} \bmod C) \\ &\cdot \dots \cdot (a^{2^{k_{i-1}}} \bmod C) \cdot (a^{2^{k_i}} \bmod C) \end{aligned}$$

$$\text{又 } A_n = a^{2^n} \bmod C$$

$$A_{n-1} = a^{2^{n-1}} \bmod C$$

$$A_n = (a^{2^{n-1}})^2 \bmod C$$

$$\therefore A_n = A_{n-1}^2 \bmod C$$

∴ 即计算后一项的结果用前一项的结果的平方取余

C代码实现:

```
int quick_pow(int a, int b, int c)
{
    int ans = 1;
    a = a % c;
    while (b > 0)
    {
        if (b & 1 == 1)
        {
            ans = (ans * a) % c;
        }
        b >>= 1;
        a = (a * a) % c;
    }
    return ans;
}
```